

定时器有几种用法

- 1.最常用的就是延迟delay
- 2.定时中断，tick中断
- 3.PWM输出控制电机的
- 4.计时计数

数组和链表的区别

- 1.数组要在一块连续的空间上，链表由节点组成。
- 2.链表的内存开销比较大
- 3.数组可以快速访问任意位置的元素，链表只能按顺序访问

队列和栈的区别

- 1.队列是先进先出，栈是先进后出

怎么传输浮点数？

```
void sendFloatAsString(UART_HandleTypeDef *huart, float value) {
    char buffer[20];
    sprintf(buffer, "%f", value);
    HAL_UART_Transmit(huart, (uint8_t*)buffer, strlen(buffer), HAL_MAX_DELAY);
}
```

主设备3.3v从设备1.8v，不使用电平转换如何通过搭电路实现通信

一般高电平的阈值电压是从设备的70%，低电压是主设备的30%，通过降低通信速率，使从设备传输数据是电平数据更加的稳定。

stm32的启动流程

- 1.MCU上电，从0x8000,0000启动(bootloader可以改)
- 2.地址为0x8000004(sp指针偏移+4)处取出复位中断向量的地址
ldr sp, [r0] ; read val from new vector
ldr r1, [r0, #4] ; read val from "new vector + 4"
- 3.复位中断服务程序执行执行main函数死循环

image-20240829201439714

如何对绝对地址0x100000赋值？

c语言
volatile uint32_t* address = (volatile uint32_t*)0x100000;
*address = value;

汇编
LDR R0, =0x100000 ; 将地址0x100000加载到寄存器R0
LDR R1, =value ; 将value加载到寄存器R1
STR R1, [R0] ; 将R1的值存储到R0指向的地址

如何实现缓存？

```

#define FIFO_SIZE 16 // FIFO 缓冲区大小
// 定义 FIFO 结构体
typedef struct {
    uint8_t buffer[FIFO_SIZE];
    uint8_t readIndex;
    uint8_t writeIndex;
} FIFO;
// 初始化 FIFO
void fifoInit(FIFO* fifo) {
    fifo->readIndex = 0;
    fifo->writeIndex = 0;
}
// 向 FIFO 中写入数据
void fifowrite(FIFO* fifo, uint8_t data) {
    uint8_t nextwriteIndex = (fifo->writeIndex + 1) % FIFO_SIZE;
    if (nextwriteIndex != fifo->readIndex) {
        fifo->buffer[fifo->writeIndex] = data;
        fifo->writeIndex = nextwriteIndex;
    }
    else {
        printf("FIFO Overflow\n"); // FIFO 已满，无法写入新数据
    }
}
// 从 FIFO 中读取数据
uint8_t fifoRead(FIFO* fifo) {
    if (fifo->readIndex != fifo->writeIndex) {
        uint8_t data = fifo->buffer[fifo->readIndex];
        fifo->readIndex = (fifo->readIndex + 1) % FIFO_SIZE;
        return data;
    }
    else {
        printf("FIFO Empty\n"); // FIFO 已空，无法读取数据
        return 0; // 返回默认值或错误码
    }
}

```

说一下 volatile，你在什么场景下会使用？

1. 一般我是在全局变量前修饰
2. 用在中断处理函数中的变量，使用到的变量可能会受到中断的干扰，导致变量的值发生变化。
3. 还有我调试的时候有时会加，假如我要记录这个函数跑了多少遍，在这个函数中一个变量++，使用volatile修饰就可以这个变量每次都能加一次，不会因为两次运行的时间实在太短了被cpu优化了，只加了一次。

static修饰的函数的变量、函数以及它的作用举例？

1. 修饰过的东西，限制了作用域，只能在当前文件中用，其他文件用不了，只能通过结构体和extern申明等方式调用
2. 保存上一次函数运行的值，在程序运行的时间内，修饰过的变量初始化只会进行一次。下次函数调用这个变量时还是这个值
3. 举例Linux驱动:修饰过的函数:通过将函数定义为 static，可以隐藏函数的实现细节，只暴露给外部的接口，从而实现信息的封装。这样可以更好地保护函数的内部逻辑，提高代码的安全性。

函数指针是怎么定义的，有什么用？

```
void (*pFunc)(int, int);
```

作用：

1. 回调函数：函数指针可以作为参数传递给另一个函数，以便在需要时回调该函数。
2. 函数的动态调用：函数指针可以根据不同的条件指向不同的函数，以便在运行时动态地调用不同的函数。

中断能否传参

不能

一般都是通过全局变量方式。不过为了避免中断函数重入，我们可以在进入中断服务函数之后将中断关闭，处理完后再开中断。

中断函数是收到中断信号自动跳转的，并非通过函数调用，什么时候运行是预先不确定的，所以不支持传参数。

ADC位数精度如何计算

一个12位的ADC可以将输入的模拟信号转换为一个12位的数字值，即有4096（ 2^{12} ）个离散的量化水平。

位数精度的计算可以通过以下公式进行：

精度 (in bits) = $\log_2$ (量化水平数)

以12位的ADC为例，它具有 $2^{12} = 4096$ 个离散的量化水平。因此，它的位数精度可以通过如下公式计算：

精度 (in bits) = $\log_2 (4096) = 12 \text{ bits}$

对于一个12位ADC，如果其测量范围为0V到3.3V，则最小分辨步进值为：

最小分辨步进值 = $3.3V / (2^{12}) \approx 0.000805V$

SPI总线最多可以连接几个设备？为什么是这么多个？

四个，SPI总线的设备数量取决于片选线（CS）的数量。

队列的底层原理

队列存储数据的实质是环形缓冲区

`xQueueCreate()`

1. 在开始创建队列之前，首先要分配内存并检查是否成功。可以使用 `pvPortMalloc()` 函数来分配内存，它是 FreeRTOS 定义的内存分配函数之一。
2. 接着，需要为队列数据结构分配内存。在这个例子中，队列数据结构是一个自定义的 `Queue_t` 结构体，包括队列元素存储区、队列深度、队列互斥量和任务等待列表等。
3. 接下来，初始化队列控制块，包括任务等待列表、队列深度、队列元素大小、读写指针等属性。同时，创建队列的互斥量，用于保护共享资源的互斥访问。
4. 最后，返回队列句柄，并将其用于后续的数据传递和同步等操作。

 image-20240829201705765

`xQueueSend()` 原理

- 1.检查队列是否有空闲空间。
- 2.如果有空闲空间，则将数据拷贝到队列中，并返回 `pdPASS` 表示发送成功。
- 3.如果没有空闲空间，则根据 `xTicksToWait` 参数判断是否等待可用空间：
 - 如果 `xTicksToWait` 为 0，则立即返回错误码 `errQUEUE_FULL`。
 - 如果 `xTicksToWait` 大于 0，则将当前任务添加到等待发送`xTaskWaitingToSend`的任务列表，并将当前任务挂起，让出 CPU 时间片给其他任务执行。
- 4.当有任务从队列接收时，会检查是否有任务等待接收，如果有，则从等待接收的任务列表`xTaskWaitingToReceive`中移除一个任务并激活它。

中断概念，如何中断，硬件中断和软件中断有什么区别，RTOS中的硬中断如何工作，软中断如何工作

- 1、软件中断由RTOS API调用或设置相应的标志位来提出中断请求/软件中断的响应时间相对较慢，因为在触发软件中断后，需要进行上下文切换到内核模式，并执行中断处理函数/软件中断处理函数在内核模式下执行。它们可以访问受保护的系统资源，并执行更复杂的操作/软件中断有优先级
- 2、硬件中断由系统外部的硬件设备或外设触发/硬件中断通常具有固定的响应时间，因为它们与硬件设备直接相关，且在实际硬件设计中有严格的时序要求/硬件中断处理函数在硬件中断上下文中执行。它们通常需要快速完成，并尽可能少地干扰其他任务的执行/硬件中断具有固定的优先级，通常由硬件设备决定。在大多数情况下，硬件中断的优先级比任务调度器的优先级高。

软件中断的处理过程

- 1.中断请求，RTOS API调用或设置相应的标志位来提出中断请求
- 2.当中断请求被触发时，RTOS 将执行上下文切换，从用户模式切换到内核模式。
- 3.中断处理函数执行，在内核模式下，RTOS 将调用与软中断相关联的中断处理函数
- 4.返回到任务执行：当中断处理完成后，RTOS 将执行上下文切换，从内核模式切换回用户模式。

RTOS系统运行中硬中断发生时，RTOS系统会如何处理

- 1.发生硬件中断
- 2.响应中断，rtos系统暂停任务，保存当前任务的上下文
- 3.中断程序执行
- 4.中断程序完成，恢复现场
- 5.任务继续执行

你是如何调试的？

- 1.发生硬件中断
- 2.响应中断，rtos系统暂停任务，保存当前任务的上下文
- 3.中断程序执行
- 4.中断程序完成，恢复现场
- 5.任务继续执行

有名管道和无名管道

无名管道对于进程是有关系的，数据在管道中按照先进先出（FIFO）的顺序传递
有名管道对于进程没有要求，任意进程都可以通过打开同一个名字的有名管道来进行通信

如何进行内核同步

在 Linux 中，内核同步是指确保多个进程或线程在访问共享资源时的正确性和一致性。下面是一些常用的内核同步机制：

1. 互斥锁（**Mutex**）：互斥锁是一种简单的同步机制，用于保护临界区（即共享资源）。只有一个线程可以获得该锁，并且其他线程必须等待。可以使用 `pthread_mutex_*` 函数或 Linux 的 `mutex_lock()` 和 `mutex_unlock()` 系统调用来操作互斥锁。

2. 读写锁（**Read-Write Lock**）：读写锁用于实现共享资源的读写操作。多个线程可以同时拥有读锁，但只有一个线程可以拥有写锁。读写锁提供了更高的并发性能，适合读取频繁、写入较少的场景。可以使用 `pthread_rwlock_*` 函数或 Linux 的 `rwlock_*` 系统调用来操作读写锁。

3. 条件变量（**Condition Variable**）：条件变量用于在线程之间进行通信，允许线程等待某个条件为真。当条件不满足时，线程可以阻塞等待，直到其他线程通过发送信号或广播唤醒它们。可以使用 `pthread_cond_*` 函数或 Linux 的 `pthread_cond_*` 系统调用来操作条件变量。

4. 信号量（**Semaphore**）：信号量是一种计数器，用于控制对共享资源的访问。它可以用来限制同时访问资源的线程数量。通过原子操作对信号量进行增加或减少，线程可以获取或释放信号量。可以使用 `sem_*` 函数或 Linux 的 `sem_*` 系统调用来操作信号量。

5. 自旋锁（**Spinlock**）：自旋锁是一种忙等待的锁机制，适合于临界区很短，线程获取锁的时间较短的场景。当线程尝试获取锁时，如果锁已被占用，线程会一直循环忙等待，直到锁被释放。可以使用 `spin_lock()` 和 `spin_unlock()` 函数来操作自旋锁。

linux的中断实现

1. 初始化: 设置中断源，让它可以设置中断，设置中断控制器(屏蔽、优先级); 设置CPU总开关(使能中断)
2. 执行程序
3. 产生中断
4. CPU每执行完一条指令就检查有无
5. 发现中断，对于不同的中断，跳去不同的地址执行。

如果再次发生了中断，那么下半部被打断，先执行硬件中断的处理函数: 硬件中断处理完发现“之前已经在处理软件中断”了，就不会往下“再次开始新的处理软件中断流程”而是退出中断处理、并恢复原来的工作: 继续执行刚刚被打断的“软件处理函数”。

为什么启动时关闭icache和dcache?

启动时关闭指令缓存（**icache**）和数据缓存（**dcache**）是为了确保代码的正确执行和数据的一致性。以下是一些可能的原因：

避免缓存脏数据：在系统启动过程中，可能存在一些临时数据或代码片段被写入到缓存中。如果这些数据不是来自可靠的来源或者没有正确初始化，那么在启动后读取这些缓存中的数据可能会导致错误的结果。关闭缓存可以确保在启动阶段使用的是内存中的最新数据。

避免指令重排序：现代处理器为了提高执行效率，可能会对指令进行乱序执行或重排序。在启动阶段，指令的执行顺序可能非常重要，特别是在初始化硬件和引导加载程序时。关闭指令缓存可以确保指令按照正确的顺序执行，避免因指令重排序而导致的错误。

linux 驱动里面的 probe 函数是什么时候被调用以及它的作用

当一个设备被插入到系统中，并且与相应的驱动匹配成功时，内核会调用该驱动的 `probe` 函数

probe函数的作用：

1. 获取硬件信息，在字符型驱动程序中，一般使用 `gpiod_get->设置gpio的模式`
`input/output(gpiod_direction_input/gpiod_direction_output)`

我自己的话写驱动的话，我的中断也是在probe函数中申请的。

2. 注册设备 `device_create`

FreeRTOS的启动流程是怎么样的？

1. 初始化阶段：在系统启动时，首先进行系统的基本初始化，包括硬件初始化和内存管理初始化。
2. 创建任务：通过调用 `xTaskCreate`` 或相关函数创建任务。每个任务是 **FreeRTOS** 的基本执行单元。
3. 创建其他系统对象：包括信号量、队列、互斥量等。
4. 启动调度器：通过调用 `vTaskStartScheduler`` 启动 **FreeRTOS** 调度器。调度器将开始根据任务优先级和调度算法调度任务的执行。
5. 任务调度：调度器将按任务的优先级和时间片安排任务执行，任务在运行中可以通过调度器切换。

CAN通信协议和I2C、SPI通信协议的比较

CAN (Controller Area Network) :

应用：主要用于汽车和工业自动化领域。

速度：较低（通常高达 **1 Mbps**）。

特性：支持多主机和多从机，具有错误检测和处理机制，支持优先级和仲裁机制。

I2C (InterIntegrated Circuit) :

应用：广泛用于低速外围设备通信，如传感器、EEPROM。

速度：较低（标准模式下最高 **100 kbps**，快速模式下最高 **400 kbps**，快速模式 **Plus** 下最高 **1 Mbps**）。

特性：支持多主机和多从机，使用两条线（**SDA** 和 **SCL**），具有简单的地址选择机制。

SPI (Serial Peripheral Interface) :

应用：用于高速外围设备通信，如闪存、ADC。

速度：较高（可以达到几十 **Mbps**）。

特性：全双工通信，支持多个从设备（需要独立的 **SS** 线），四条线（**MOSI**、**MISO**、**SCK**、**SS**）。

FLASH的使用、读和写操作比较

使用：**FLASH** 存储是一种非易失性存储器，用于存储程序代码和配置数据。它在嵌入式系统中广泛使用。

读操作：**FLASH** 读操作相对较快，通常不需要额外的操作时间。

写操作：**FLASH** 写操作较慢，并且通常需要擦除操作来重写数据。写入新的数据之前，必须先擦除相应的存储块。

软中断和硬中断的概念

硬中断：由硬件事件触发，通常是外部设备发出的信号。中断处理程序（**ISR**）在硬件中断信号到达时立即响应。

软中断：由软件触发，通常用于实现系统调用或任务调度。它是通过软件指令生成的中断请求，通常用于在操作系统中执行特定操作。

定时器有几种用法

数组和链表的区别

队列和栈的区别

怎么传输浮点数？

主设备3.3v从设备1.8v，不使用电平转换如何通过搭电路实现通信

stm32的启动流程

如何对绝对地址0x100000赋值？

如何实现缓存？

说一下 volatile，你在什么场景下会使用？

static修饰的函数的变量、函数以及它的作用举例？

函数指针是怎么定义的，有什么用？

中断能否传参

ADC位数精度如何计算

SPI总线最多可以连接几个设备？为什么是这么多个？

RTOS队列的底层原理

xQueueSend() 原理

中断概念，如何中断，硬件中断和软件中断有什么区别，RTOS中的硬中断如何工作，软中断如何工作

RTOS系统运行中硬中断发生时，RTOS系统会如何处理

你是如何调试的？

有名管道和无名管道

如何进行内核同步

linux的中断实现

为什么启动时关闭icache和dcache？

linux 驱动里面的 probe 函数是什么时候被调用以及它的作用

CAN通信协议和I2C、SPI通信协议的比较

FreeRTOS的启动流程是怎么样的？

FLASH的使用、读和写操作比较

软中断和硬中断的概念